

Bank Management System PL/SQL

Aim

To implement banking operations using MySQL stored procedures, functions, triggers and cursors.

Database Creation

```
DROP DATABASE IF EXISTS BankDB;
```

```
CREATE DATABASE BankDB;
```

```
USE BankDB;
```

```
SHOW DATABASES;
```

Table Creation

```
USE bankdb;
```

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    DOB DATE,  
    Balance DECIMAL(10,2),  
    LastModified DATETIME,  
    IsVIP BOOLEAN DEFAULT FALSE  
);
```

```
CREATE TABLE Accounts (  
    AccountID INT PRIMARY KEY,  
    CustomerID INT,  
    AccountType VARCHAR(20),  
    Balance DECIMAL(10,2),
```

```
LastModified DATETIME,  
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE Transactions (  
    TransactionID INT PRIMARY KEY,  
    AccountID INT,  
    TransactionDate DATETIME,  
    Amount DECIMAL(10,2),  
    TransactionType VARCHAR(20),  
    FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)  
);
```

```
CREATE TABLE Loans (  
    LoanID INT PRIMARY KEY,  
    CustomerID INT,  
    LoanAmount DECIMAL(10,2),  
    InterestRate DECIMAL(5,2),  
    StartDate DATE,  
    EndDate DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    Name VARCHAR(100),
```

```
Position VARCHAR(50),  
Salary DECIMAL(10,2),  
Department VARCHAR(50),  
HireDate DATE  
);
```

```
CREATE TABLE AuditLog (  
    LogID INT AUTO_INCREMENT PRIMARY KEY,  
    TransactionID INT,  
    AccountID INT,  
    Amount DECIMAL(10,2),  
    TransactionType VARCHAR(20),  
    LogDate DATETIME  
);
```

Sample Data Insertion

```
USE bankdb;
```

```
-- Customers
```

```
INSERT INTO Customers VALUES  
(1,'John Doe','1960-05-15',15000,NOW(),FALSE),  
(2,'Jane Smith','1990-07-20',8000,NOW(),FALSE),  
(3,'Robert','1955-08-10',25000,NOW(),FALSE);
```

```
-- Accounts
```

```
INSERT INTO Accounts VALUES
```

```
(101,1,'Savings',15000,NOW()),  
(102,2,'Checking',8000,NOW()),  
(103,3,'Savings',25000,NOW());
```

-- Transactions

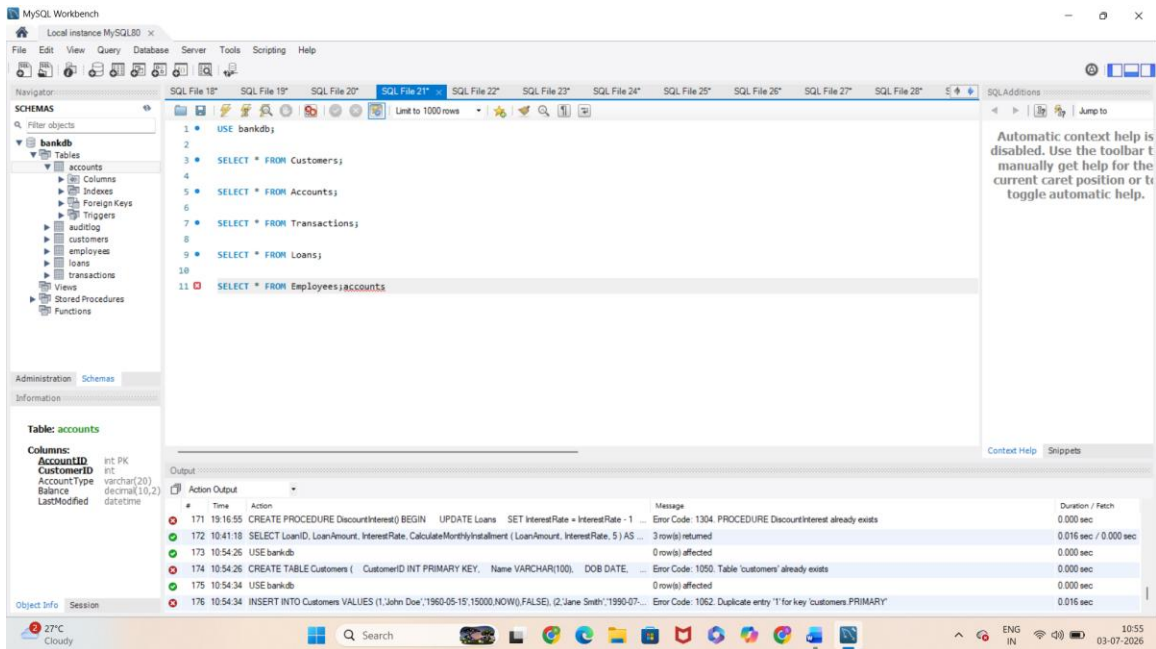
```
INSERT INTO Transactions VALUES  
(1,101,NOW(),500,'Deposit'),  
(2,102,NOW(),300,'Withdrawal');
```

-- Loans

```
INSERT INTO Loans VALUES  
(1,1,50000,10,CURDATE(),DATE_ADD(CURDATE(),INTERVAL 20 MONTH)),  
(2,2,30000,11,CURDATE(),DATE_ADD(CURDATE(),INTERVAL 2 MONTH)),  
(3,3,45000,9,CURDATE(),DATE_ADD(CURDATE(),INTERVAL 1 MONTH));
```

-- Employees

```
INSERT INTO Employees VALUES  
(1,'Alice Johnson','Manager',70000,'HR','2015-06-15'),  
(2,'Bob Brown','Developer',60000,'IT','2017-03-20');
```



Exercise 1: Control Structures

Scenario 1: Discount Interest

Requirement:

Apply 1% discount to customers above 60 years.

Code

USE bankdb;

DROP PROCEDURE IF EXISTS DiscountInterest;

DELIMITER \$\$

```

CREATE PROCEDURE DiscountInterest()
BEGIN

```

```

UPDATE Loans
SET InterestRate = InterestRate - 1
WHERE CustomerID IN
(
SELECT CustomerID

```

```

FROM
(
SELECT CustomerID
FROM Customers
WHERE TIMESTAMPDIFF(YEAR,DOB,CURDATE()) > 60
) AS temp
);

SELECT '1% Interest Discount Applied Successfully' AS Message;

END$$

```

DELIMITER ;

Execution Query

```
USE bankdb;
```

```
CALL DiscountInterest();
```

Verification Query

```
SELECT * FROM Loans;
```

Output

The top screenshot shows a 'Result Grid' with a single row containing the message '1% Interest Discount Applied Successfully'.

The bottom screenshot shows a 'Result Grid' with a table of loans. The table has three columns: LoanID, CustomerID, and InterestRate. The data is as follows:

LoanID	CustomerID	InterestRate
1	1	9.00
2	2	11.00
3	3	8.00

Below the table, there is an 'Action Output' section showing two rows of execution logs:

#	Time	Action	Message
166	19:04:16	SELECT * FROM Employees LIMIT 0, 1000	3 row(s) returned
167	19:04:16	SELECT * FROM Loans LIMIT 0, 1000	3 row(s) returned

Scenario 2: VIP Customers

Requirement

Update IsVIP for balance >10000.

Code

```
USE bankdb;
```

```
DROP PROCEDURE IF EXISTS UpdateVIPCustomers;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE UpdateVIPCustomers()  
BEGIN
```

```
    UPDATE Customers  
    SET IsVIP = TRUE  
    WHERE Balance > 10000;
```

```
    SELECT 'VIP Status Updated Successfully' AS Message;
```

```
END$$
```

```
DELIMITER ;
```

Execution Query

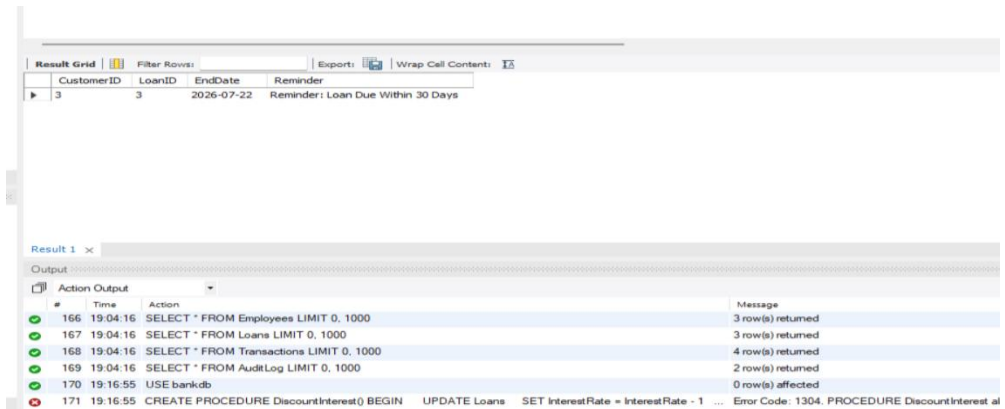
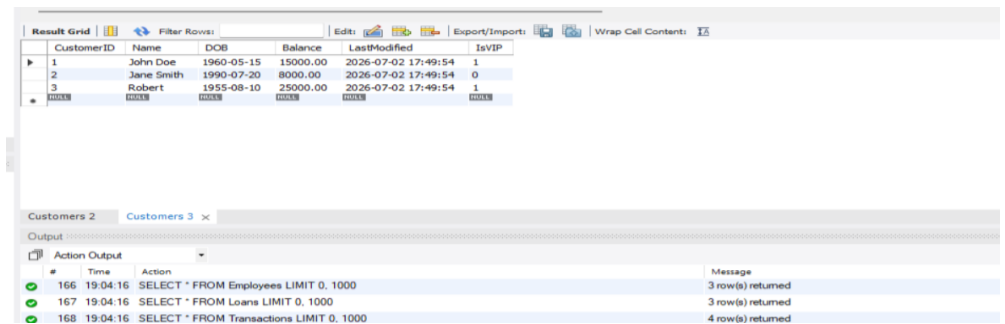
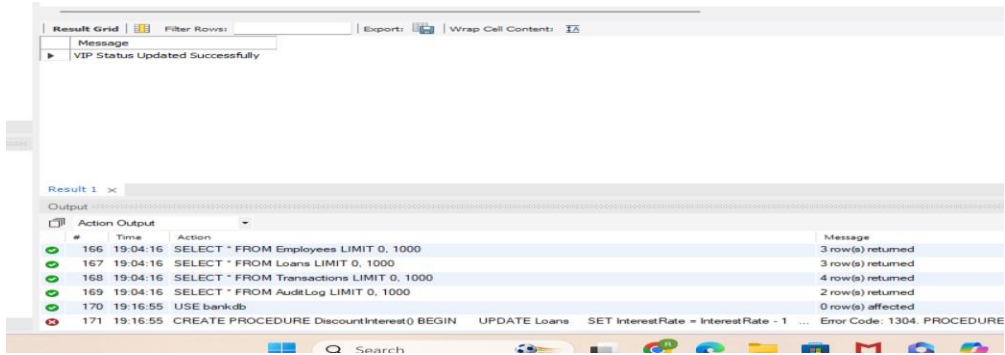
```
USE bankdb;
```

```
CALL UpdateVIPCustomers();
```

Verification Query

```
SELECT *  
FROM Customers;
```

Output



Scenario 3: Loan Reminder

Requirement

Show loans due within 30 days.

Code


```
USE bankdb;

DROP PROCEDURE IF EXISTS LoanReminder;

DELIMITER $$

CREATE PROCEDURE LoanReminder()
BEGIN

SELECT
CustomerID,
LoanID,
EndDate,
'Reminder: Loan Due Within 30 Days' AS Reminder

FROM Loans

WHERE EndDate
BETWEEN CURDATE()

AND DATE_ADD
(
CURDATE(),
INTERVAL 30 DAY
);

END$$

DELIMITER ;
```

Execution Query

```
USE bankdb;

CALL LoanReminder();
```

Verification Query

```
SELECT *
FROM Loans;
```

Output

Result Grid				
Filter Rows:				
AccountID	CustomerID	AccountType	Balance	LastModified
101	1	Savings	15000.00	2026-07-02 17:49:54
102	2	Checking	8000.00	2026-07-02 17:49:54
103	3	Savings	25000.00	2026-07-02 17:49:54
NULL	NULL	NULL	NULL	NULL

Result Grid	
Filter Rows:	
Export:	
Wrap Cell Content:	
Message	
Fund Transfer Successful	

Result Grid				
Filter Rows:				
AccountID	Balance			
101	14000.00			
102	9000.00			
103	25000.00			
NULL	NULL			

Accounts 4			
Output			
#	Time	Action	Message
166	19:04:16	SELECT * FROM Employees LIMIT 0, 1000	3 row(s) returned
167	19:04:16	SELECT * FROM Loans LIMIT 0, 1000	3 row(s) returned
168	19:04:16	SELECT * FROM Transfers LIMIT 0, 1000	3 row(s) returned

Result Grid				
Filter Rows:				
AccountID	Balance			
101	14000.00			
102	9000.00			
103	25000.00			
NULL	NULL			

Accounts 1	
Output	
#	Action

Exercise 2: Error Handling

Scenario 1: SafeTransferFunds

Requirement:

Transfer funds with rollback.

Code

USE bankdb;

DROP PROCEDURE IF EXISTS SafeTransferFunds;

DELIMITER \$\$

```
CREATE PROCEDURE SafeTransferFunds(  
    IN FromAccount INT,  
    IN ToAccount INT,  
    IN TransferAmount DECIMAL(10,2)  
)  
BEGIN  
  
    DECLARE SourceBalance DECIMAL(10,2);  
  
    START TRANSACTION;  
  
    SELECT Balance  
    INTO SourceBalance  
    FROM Accounts  
    WHERE AccountID = FromAccount;  
  
    IF SourceBalance >= TransferAmount THEN  
  
        UPDATE Accounts  
        SET Balance = Balance - TransferAmount  
        WHERE AccountID = FromAccount;  
  
        UPDATE Accounts  
        SET Balance = Balance + TransferAmount  
        WHERE AccountID = ToAccount;  
  
        COMMIT;  
  
        SELECT 'Fund Transfer Successful' AS Message;  
  
    ELSE
```

ROLLBACK;

SELECT 'Insufficient Funds! Transaction Rolled Back.' AS Message;

END IF;

END\$\$

DELIMITER ;

Execution Query

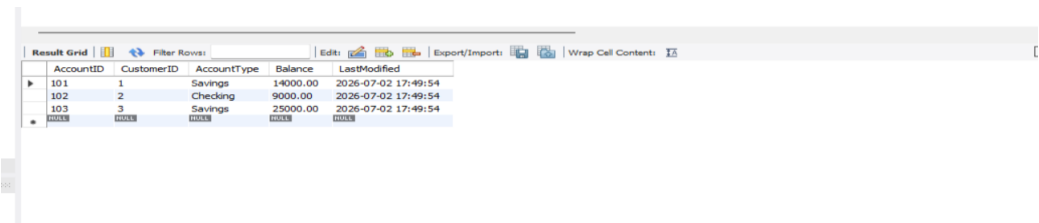
USE bankdb;

CALL SafeTransferFunds(101,102,1000);

Verification Query

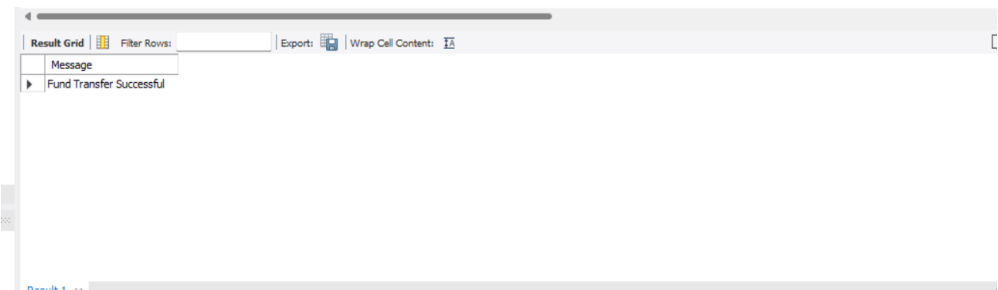
SELECT *
FROM Accounts;

Output:



A screenshot of a database application's 'Result Grid' window. The window has a toolbar with icons for 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. Below the toolbar is a table with five columns: 'AccountID', 'CustomerID', 'AccountType', 'Balance', and 'LastModified'. The table contains three rows of data.

AccountID	CustomerID	AccountType	Balance	LastModified
101	1	Savings	14000.00	2026-07-02 17:49:54
102	2	Checking	9000.00	2026-07-02 17:49:54
103	3	Savings	25000.00	2026-07-02 17:49:54



A screenshot of a database application's 'Result Grid' window. The window has a toolbar with icons for 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the toolbar is a table with one column: 'Message'. The table contains one row of data.

Message
Fund Transfer Successful

The screenshot shows a database application window. At the top, there's a 'Result Grid' tab and a 'Filter Rows' input field. Below this is a table with two columns: 'AccountID' and 'Balance'. The table contains three rows of data. To the right of the table is a toolbar with icons for 'Edit', 'Export/Import', and 'Wrap Cell Content'. On the far right, there's a vertical sidebar with buttons for 'Result Grid', 'Form Editor', and 'Field Types'. At the bottom of the window, there's a status bar showing 'Accounts 1' and buttons for 'Apply' and 'Revert'.

AccountID	Balance
101	13000.00
102	10000.00
103	25000.00

Scenario 2: UpdateSalary

Requirement:

Increase employee salary.

Code

USE bankdb;

DROP PROCEDURE IF EXISTS UpdateSalary;

DELIMITER \$\$

CREATE PROCEDURE UpdateSalary

```
(
  IN EmpID INT,
  IN Percentage DECIMAL(5,2)
)
BEGIN
```

IF EXISTS

```
(
  SELECT *
  FROM Employees
  WHERE EmployeeID = EmpID
)
THEN
```

UPDATE Employees

```
SET Salary = Salary +
    (Salary * Percentage /100)
```

WHERE EmployeeID = EmpID;

```
SELECT 'Salary Updated Successfully'  
AS Message;
```

```
ELSE
```

```
SELECT 'Employee ID Not Found'  
AS Message;
```

```
END IF;
```

```
END$$
```

```
DELIMITER ;
```

Execution Query

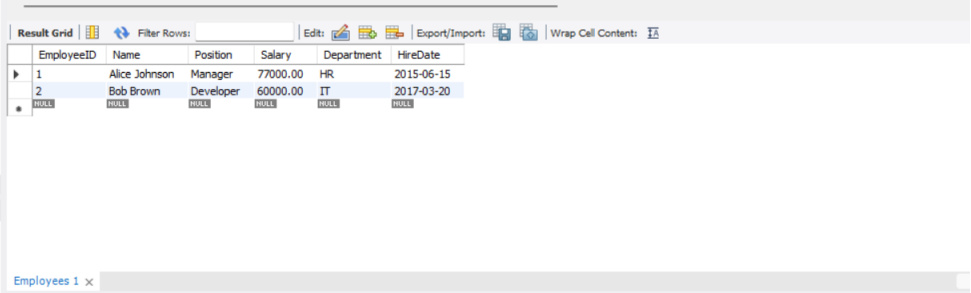
```
USE bankdb;
```

```
CALL UpdateSalary(1,10);
```

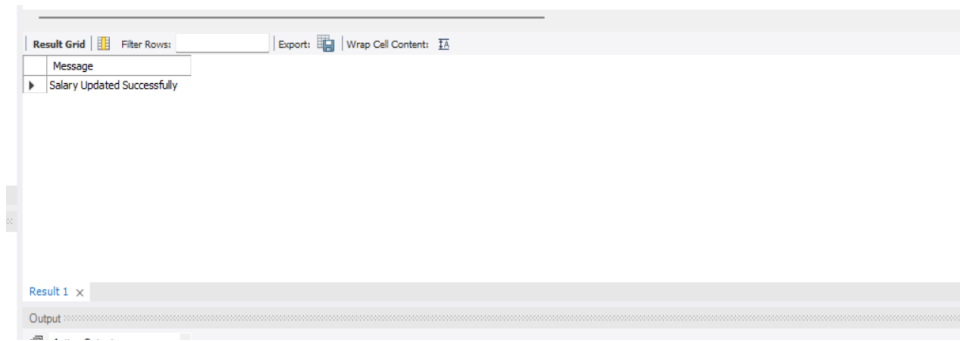
Verification Query

```
SELECT *  
FROM Employees;
```

Output



EmployeeID	Name	Position	Salary	Department	HireDate
1	Alice Johnson	Manager	77000.00	HR	2015-06-15
2	Bob Brown	Developer	60000.00	IT	2017-03-20



Scenario 3: AddNewCustomer

Requirement:

Insert customer with duplicate handling.

Code

USE bankdb;

DROP PROCEDURE IF EXISTS AddNewCustomer;

DELIMITER \$\$

CREATE PROCEDURE AddNewCustomer

```
(  
    IN CID INT,  
    IN CName VARCHAR(100),  
    IN CDOB DATE,  
    IN CBalance DECIMAL(10,2)  
)
```

BEGIN

```
    IF EXISTS  
    (  
        SELECT *  
        FROM Customers  
        WHERE CustomerID = CID  
    )
```

THEN

SELECT 'Customer ID Already Exists'
AS Message;

ELSE

INSERT INTO Customers

(
CustomerID,
Name,
DOB,
Balance,
LastModified,
IsVIP
)

VALUES

(
CID,
CName,
CDOB,
CBalance,
NOW(),
FALSE
);

SELECT 'Customer Added Successfully'
AS Message;

END IF;

END\$\$

DELIMITER ;

Execution Query

USE bankdb;

CALL AddNewCustomer

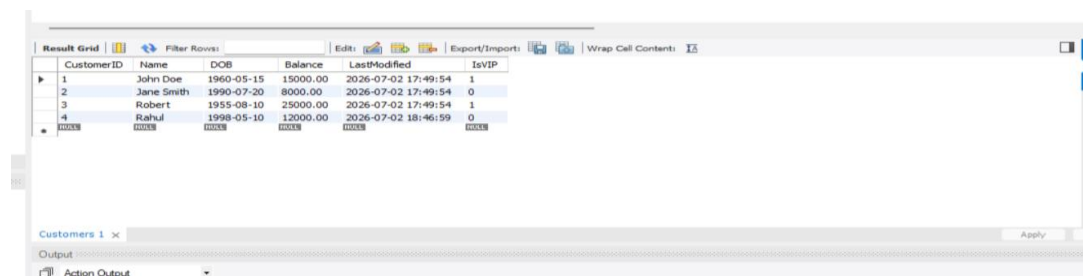
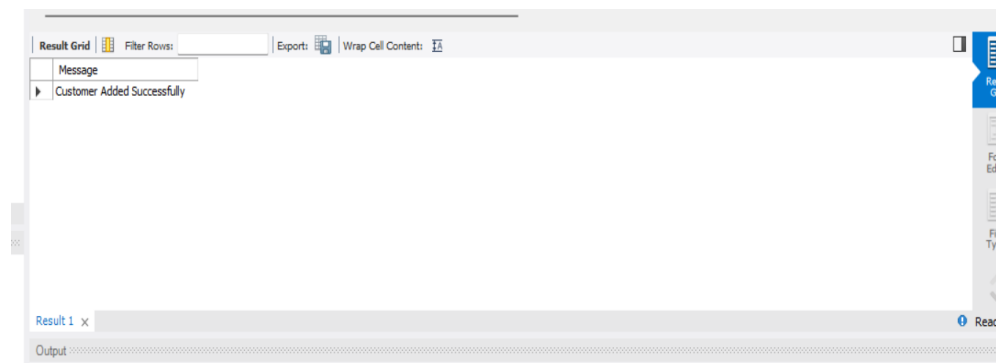
(
4,
'Rahul',


```
'1998-05-10',  
12000  
);
```

Verification Query

```
SELECT *  
FROM Customers;
```

Output



Exercise 3: Stored Procedures

Scenario 1: ProcessMonthlyInterest

Requirement:

Apply 1% interest to all Savings accounts.

Code

```
USE bankdb;
```

```
DROP PROCEDURE IF EXISTS ProcessMonthlyInterest;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE ProcessMonthlyInterest()  
BEGIN
```

```
    UPDATE Accounts  
    SET Balance = Balance + (Balance * 0.01)  
    WHERE AccountType = 'Savings';
```

```
    SELECT 'Monthly Interest Applied Successfully' AS Message;
```

```
END$$
```

```
DELIMITER ;
```

Execution Query

```
USE bankdb;
```

```
CALL ProcessMonthlyInterest();
```

Verification Query

```
SELECT *  
FROM Accounts;
```

Output

Result Grid Filter Rows: Export: Wrap Cell Contents:					
Message					
▶ Monthly Interest Applied Successfully					
Result 1 x					
Output					
Action Output					
#	Time	Action	Message		
166	19:04:16	SELECT * FROM Employees LIMIT 0, 1000	3 row(s) returned		
167	19:04:16	SELECT * FROM Loans LIMIT 0, 1000	3 row(s) returned		
168	19:04:16	SELECT * FROM Transactions LIMIT 0, 1000	4 row(s) returned		
169	19:04:16	SELECT * FROM AuditLog LIMIT 0, 1000	2 row(s) returned		

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Contents:					
	AccountID	CustomerID	AccountType	Balance	LastModified
▶	101	1	Savings	13130.00	2026-07-02 17:49:54
	102	2	Checking	10000.00	2026-07-02 17:49:54
	103	3	Savings	25250.00	2026-07-02 17:49:54
*	NULL	NULL	NULL	NULL	NULL

Accounts 1 x					
Output					
Action Output					
#	Time	Action	Message		
166	19:04:16	SELECT * FROM Employees LIMIT 0, 1000	3 row(s) returned		
167	19:04:16	SELECT * FROM Loans LIMIT 0, 1000	3 row(s) returned		

Scenario 2: UpdateEmployeeBonus

Requirement:

Update employee salary by adding a bonus percentage.

Code

USE bankdb;

DROP PROCEDURE IF EXISTS UpdateEmployeeBonus;

DELIMITER \$\$

CREATE PROCEDURE UpdateEmployeeBonus

(
 IN Dept VARCHAR(50),
 IN BonusPercent DECIMAL(5,2)
)

BEGIN

```
UPDATE Employees
```

```
SET Salary = Salary +  
    (Salary * BonusPercent / 100)
```

```
WHERE Department = Dept;
```

```
SELECT 'Employee Bonus Updated Successfully'  
AS Message;
```

```
END$$
```

```
DELIMITER ;
```

Execution Query

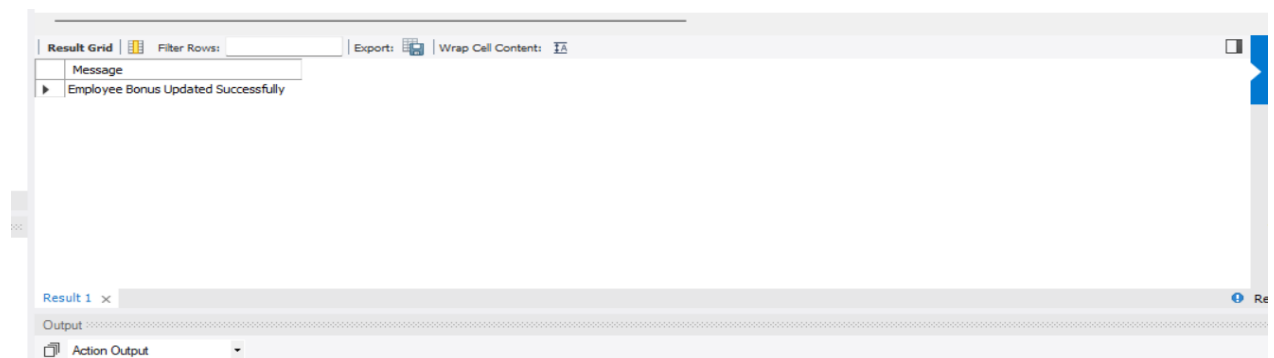
```
USE bankdb;
```

```
CALL UpdateEmployeeBonus('HR',10);
```

Verification Query

```
SELECT *  
FROM Employees;
```

Output



Result Grid						
Filter Rows:						
Edit: Export/Import: Wrap Cell Content:						
EmployeeID	Name	Position	Salary	Department	HireDate	
1	Alice Johnson	Manager	84700.00	HR	2015-06-15	
2	Bob Brown	Developer	60000.00	IT	2017-03-20	
...	...	...	...	...	...	

Scenario 3: TransferFunds

Requirement:

Transfer funds between two accounts after checking sufficient balance.

Code

USE bankdb;

DROP PROCEDURE IF EXISTS TransferFunds;

DELIMITER \$\$

CREATE PROCEDURE TransferFunds

```
(
  IN SourceAccount INT,
  IN DestinationAccount INT,
  IN Amount DECIMAL(10,2)
)
```

BEGIN

 DECLARE CurrentBalance DECIMAL(10,2);

```
    SELECT Balance
    INTO CurrentBalance
    FROM Accounts
    WHERE AccountID = SourceAccount;
```

 IF CurrentBalance >= Amount THEN

 UPDATE Accounts

```
SET Balance = Balance - Amount
WHERE AccountID = SourceAccount;
```

```
UPDATE Accounts
SET Balance = Balance + Amount
WHERE AccountID = DestinationAccount;
```

```
SELECT 'Transfer Completed Successfully'
AS Message;
```

```
ELSE
```

```
SELECT 'Insufficient Balance'
AS Message;
```

```
END IF;
```

```
END$$
```

```
DELIMITER ;
```

Execution Query

```
USE bankdb;
```

```
CALL TransferFunds
(
101,
103,
2000
);
```

Verification Query

```
SELECT *
FROM Accounts;
```

Output

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Message

Transfer Completed Successfully

Result 1

Output

Action Output

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

AccountID	CustomerID	AccountType	Balance	LastModified
101	1	Savings	11130.00	2026-07-02 17:49:54
102	2	Checking	10000.00	2026-07-02 17:49:54
103	3	Savings	27250.00	2026-07-02 17:49:54
NULL	NULL	NULL	NULL	NULL

Accounts 1

Exercise 4: Functions

Scenario 1: CalculateAge

Requirement:

Return the customer's age based on Date of Birth (DOB).

Code

```
USE bankdb;
```

```
DROP FUNCTION IF EXISTS CalculateAge;
```

```
DELIMITER $$
```

```
CREATE FUNCTION CalculateAge
```

```
(  
    DOB DATE  
)
```

```
RETURNS INT
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    DECLARE Age INT;
```

```
    SET Age = TIMESTAMPDIFF(YEAR, DOB, CURDATE());
```

```
    RETURN Age;
```

```
END$$
```

```
DELIMITER ;
```

Execution Query

```
USE bankdb;
```

```
SELECT
```



```
CustomerID,  
Name,  
CalculateAge(DOB) AS Age  
FROM Customers;
```

Verification Query

```
SELECT *  
FROM Customers;
```

Output

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	CustomerID	Name	Age
▶	1	John Doe	66
	2	Jane Smith	35
	3	Robert	70
	4	Rahul	28

Result 1

×

Output

Action Output

▼

#	Time	Action	Message
---	------	--------	---------

Scenario 2: CalculateMonthlyInstallment

Requirement:

Calculate the monthly loan installment (EMI).

Code

```
USE bankdb;
```

```
DROP FUNCTION IF EXISTS CalculateMonthlyInstallment;
```

```
DELIMITER $$
```

```
CREATE FUNCTION CalculateMonthlyInstallment  
(  
    LoanAmount DECIMAL(10,2),  
    InterestRate DECIMAL(5,2),  
    Years INT
```

)

RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

DECLARE EMI DECIMAL(10,2);

SET EMI =

(
LoanAmount +
(LoanAmount * InterestRate * Years / 100)
)/
(Years * 12);

RETURN EMI;

END\$\$

DELIMITER ;

Execution Query

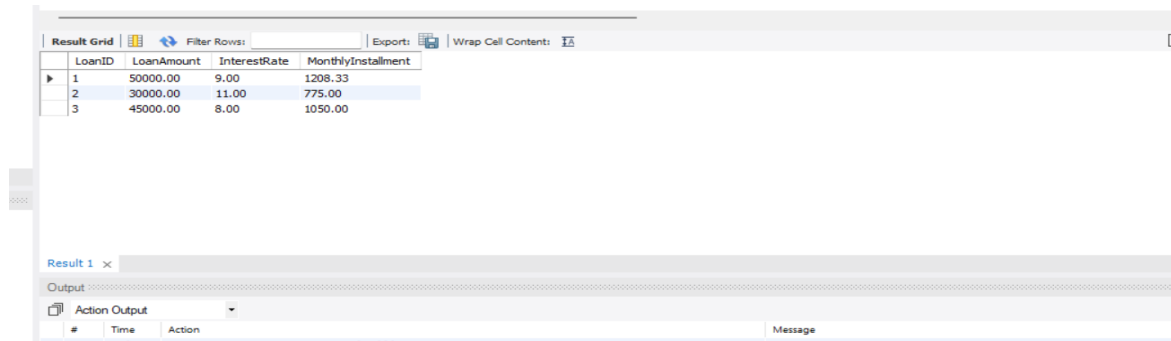
USE bankdb;

SELECT
LoanID,
LoanAmount,
InterestRate,
CalculateMonthlyInstallment
(
LoanAmount,
InterestRate,
5
)
AS MonthlyInstallment
FROM Loans;

Verification Query

SELECT *
FROM Loans;

Output



The screenshot shows a software interface with a 'Result Grid' at the top and an 'Output' pane at the bottom. The 'Result Grid' contains the following data:

LoanID	LoanAmount	InterestRate	MonthlyInstallment
1	50000.00	9.00	1208.33
2	30000.00	11.00	775.00
3	45000.00	8.00	1050.00

The 'Output' pane is currently empty, showing a header with columns: #, Time, Action, and Message.

Scenario 3: HasSufficientBalance

Requirement:

Return TRUE or FALSE depending on whether an account has sufficient balance.

Code

USE bankdb;

DROP FUNCTION IF EXISTS HasSufficientBalance;

DELIMITER \$\$

```
CREATE FUNCTION HasSufficientBalance
(
    AccountNo INT,
    Amount DECIMAL(10,2)
)
```

RETURNS BOOLEAN

DETERMINISTIC

BEGIN

 DECLARE CurrentBalance DECIMAL(10,2);

```
    SELECT Balance
    INTO CurrentBalance
    FROM Accounts
```

```
WHERE AccountID = AccountNo;
```

```
RETURN CurrentBalance >= Amount;
```

```
END$$
```

```
DELIMITER ;
```

Execution Query

```
USE bankdb;
```

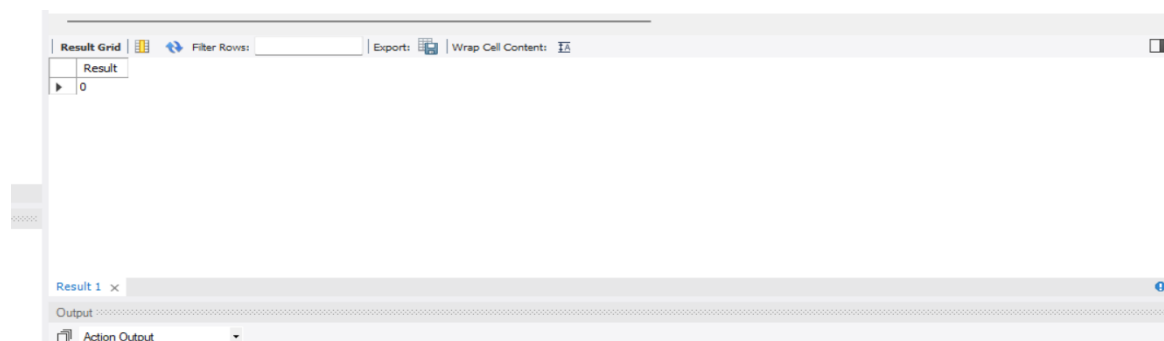
```
SELECT  
HasSufficientBalance  
(  
101,  
5000  
)  
AS Result;
```

```
SELECT  
HasSufficientBalance  
(  
102,  
50000  
)  
AS Result;
```

Verification Query

```
SELECT *  
FROM Accounts;
```

Output



Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	AccountID	CustomerID	AccountType	Balance	LastModified
▶	101	1	Savings	11130.00	2026-07-02 17:49:54
	102	2	Checking	10000.00	2026-07-02 17:49:54
	103	3	Savings	27250.00	2026-07-02 17:49:54
•	NULL	NULL	NULL	NULL	NULL

Exercise 5: Triggers

Scenario 1: UpdateCustomerLastModified

Requirement:

Automatically update the **LastModified** column whenever a customer's record is updated.

Code

```
USE bankdb;
```

```
DROP TRIGGER IF EXISTS UpdateCustomerLastModified;
```

```
DELIMITER $$
```

```
CREATE TRIGGER UpdateCustomerLastModified
```

```
BEFORE UPDATE
```

```
ON Customers
```

```
FOR EACH ROW
```

```
BEGIN
```

```
SET NEW.LastModified = NOW();
```

```
END$$
```

```
DELIMITER ;
```

Execution Query

```
UPDATE Customers
```

```
SET Balance = 18000
```

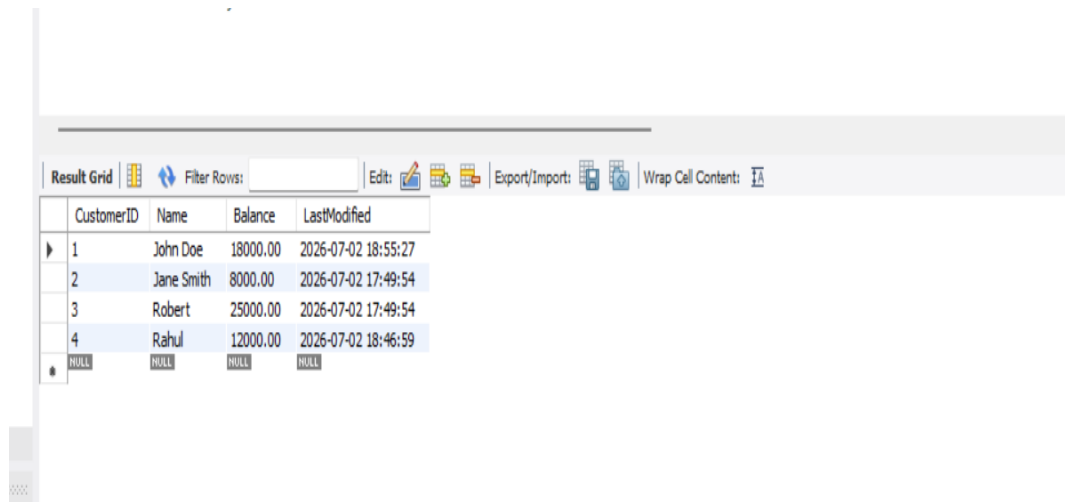
```
WHERE CustomerID = 1;
```

Verification Query

```
SELECT *
```

```
FROM Customers;
```

Output



The screenshot shows a database application interface. At the top, there is a toolbar with icons for 'Result Grid', 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. Below the toolbar is a table with the following data:

	CustomerID	Name	Balance	LastModified
▶	1	John Doe	18000.00	2026-07-02 18:55:27
	2	Jane Smith	8000.00	2026-07-02 17:49:54
	3	Robert	25000.00	2026-07-02 17:49:54
	4	Rahul	12000.00	2026-07-02 18:46:59
✱	NULL	NULL	NULL	NULL

Scenario 2: LogTransaction

Requirement:

Maintain an audit log whenever a transaction is inserted.

Code

USE bankdb;

DROP TRIGGER IF EXISTS LogTransaction;

DELIMITER \$\$

CREATE TRIGGER LogTransaction

AFTER INSERT

ON Transactions

FOR EACH ROW

BEGIN

INSERT INTO AuditLog

(
TransactionID,
AccountID,
Amount,

```
TransactionType,  
LogDate  
)
```

```
VALUES  
(  
NEW.TransactionID,  
NEW.AccountID,  
NEW.Amount,  
NEW.TransactionType,  
NOW()  
);
```

```
END$$
```

```
DELIMITER ;
```

Execution Query

```
INSERT INTO Transactions  
VALUES  
(  
3,  
101,  
NOW(),  
1500,  
'Deposit'  
);
```

Verification Query

```
SELECT *  
FROM AuditLog;
```

Output

The screenshot shows a SQL Server Enterprise Manager window. The 'Result Grid' pane displays a single row of data from a query. The columns are LogID, TransactionID, AccountID, Amount, TransactionType, and LogDate. The data row shows LogID 1, TransactionID 3, AccountID 101, Amount 1500.00, TransactionType Deposit, and LogDate 2026-07-02 18:56:00. Below the Result Grid is a 'Messages' pane with a tab labeled 'AuditLog 1'. The 'Output' pane is also visible at the bottom.

LogID	TransactionID	AccountID	Amount	TransactionType	LogDate
1	3	101	1500.00	Deposit	2026-07-02 18:56:00

Scenario 3: CheckTransactionRules

Requirement:

Ensure deposits are positive and withdrawals do not exceed the account balance.

Code

```
USE bankdb;
```

```
DROP TRIGGER IF EXISTS CheckTransactionRules;
```

```
DELIMITER $$
```

```
CREATE TRIGGER CheckTransactionRules
```

```
BEFORE INSERT
```

```
ON Transactions
```

```
FOR EACH ROW
```

```
BEGIN
```

```
DECLARE CurrentBalance DECIMAL(10,2);
```

```
IF NEW.TransactionType='Deposit'
```

```
AND NEW.Amount<=0 THEN
```

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT='Deposit Amount Must Be Greater Than Zero';
```

```
END IF;
```

```
IF NEW.TransactionType='Withdrawal' THEN
```

```
SELECT Balance  
INTO CurrentBalance  
FROM Accounts  
WHERE AccountID=NEW.AccountID;
```

```
IF NEW.Amount>CurrentBalance THEN
```

```
SIGNAL SQLSTATE '45000'  
SET MESSAGE_TEXT='Insufficient Balance';
```

```
END IF;
```

```
END IF;
```

```
END$$
```

```
DELIMITER ;
```

Execution Query

```
INSERT INTO Transactions  
VALUES  
(  
4,  
101,  
NOW(),  
-500,  
'Deposit'  
);
```

```
INSERT INTO Transactions  
VALUES  
(  
5,  
102,  
NOW(),  
50000,
```

'Withdrawal'

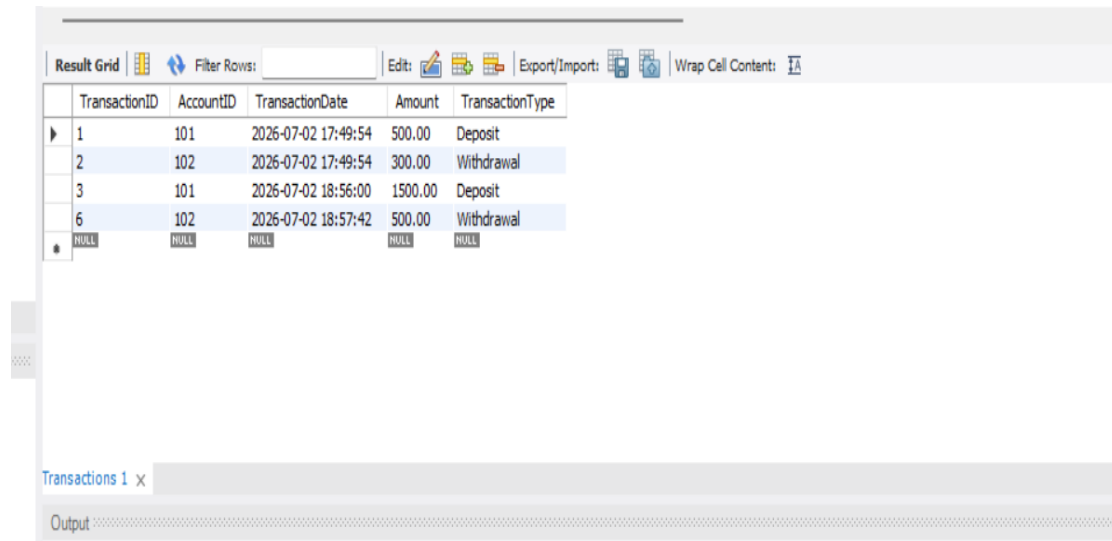
);

Verification Query

SELECT *

FROM Transactions;

Output



The screenshot shows a database query result grid with a toolbar at the top. The toolbar includes buttons for 'Result Grid', 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. The grid displays a table with the following data:

	TransactionID	AccountID	TransactionDate	Amount	TransactionType
▶	1	101	2026-07-02 17:49:54	500.00	Deposit
	2	102	2026-07-02 17:49:54	300.00	Withdrawal
	3	101	2026-07-02 18:56:00	1500.00	Deposit
	6	102	2026-07-02 18:57:42	500.00	Withdrawal
•	NULL	NULL	NULL	NULL	NULL

Below the grid, there is a tab labeled 'Transactions 1' and an 'Output' section.

Exercise 6: Cursors

Scenario 1: GenerateMonthlyStatements

Requirement:

Generate monthly statements for all customers using an explicit cursor.

Code

```
USE bankdb;
```

```
DROP PROCEDURE IF EXISTS GenerateMonthlyStatements;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE GenerateMonthlyStatements()
```

```
BEGIN
```

```
DECLARE Done INT DEFAULT FALSE;
```

```
DECLARE TID INT;
```

```
DECLARE AID INT;
```

```
DECLARE Amt DECIMAL(10,2);
```

```
DECLARE TType VARCHAR(20);
```

```
DECLARE cur CURSOR FOR
```

```
SELECT TransactionID,
```

```
AccountID,
```

```
Amount,
```

```
TransactionType
```

```
FROM Transactions
```

```
WHERE MONTH(TransactionDate)=MONTH(CURDATE())
```

```
AND YEAR(TransactionDate)=YEAR(CURDATE());
```

```
DECLARE CONTINUE HANDLER
```

```
FOR NOT FOUND
```

```
SET Done=TRUE;

OPEN cur;

read_loop: LOOP

FETCH cur INTO TID,AID,Amt,TType;

IF Done THEN
LEAVE read_loop;
END IF;

SELECT
TID,
AID,
Amt,
TType;

END LOOP;

CLOSE cur;

END$$

DELIMITER ;
```

Execution Query

```
CALL GenerateMonthlyStatements();
```

Verification Query

```
SELECT *
FROM Transactions;
```

Output

TransactionID	AccountID	Amount	TransactionType
6	102	500.00	Withdrawal

#	Time	Action	Message
1	10:04:10	SELECT FROM ... LIMIT 0, 1000	Success

Scenario 2: ApplyAnnualFee

Requirement:

Deduct ₹500 annual maintenance fee from all accounts.

Code

USE bankdb;

DROP PROCEDURE IF EXISTS ApplyAnnualFee;

DELIMITER \$\$

CREATE PROCEDURE ApplyAnnualFee()

BEGIN

DECLARE Done INT DEFAULT FALSE;

DECLARE AccID INT;

DECLARE cur CURSOR FOR

SELECT AccountID
FROM Accounts;

DECLARE CONTINUE HANDLER
FOR NOT FOUND
SET Done=TRUE;

OPEN cur;

read_loop: LOOP

FETCH cur INTO AccID;

IF Done THEN

LEAVE read_loop;

END IF;

UPDATE Accounts

SET Balance=Balance-500

WHERE AccountID=AccID;

END LOOP;

CLOSE cur;

SELECT 'Annual Fee Applied Successfully'

AS Message;

END\$\$

DELIMITER ;

Execution Query

CALL ApplyAnnualFee();

Verification Query

SELECT *

FROM Accounts;

Output

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	AccountID	CustomerID	AccountType	Balance	LastModified
▶	101	1	Savings	10630.00	2026-07-02 17:49:54
	102	2	Checking	9500.00	2026-07-02 17:49:54
	103	3	Savings	26750.00	2026-07-02 17:49:54
•	NULL	NULL	NULL	NULL	NULL

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Message

▶ Annual Fee Applied Successfully

Result 1 ×

Output

Action Output

Scenario 3: UpdateLoanInterestRates

Requirement:

Update loan interest rates using an explicit cursor.

Code

USE bankdb;

DROP PROCEDURE IF EXISTS UpdateLoanInterestRates;

DELIMITER \$\$

CREATE PROCEDURE UpdateLoanInterestRates()

BEGIN

DECLARE Done INT DEFAULT FALSE;


```
DECLARE LID INT;

DECLARE cur CURSOR FOR

SELECT LoanID
FROM Loans;

DECLARE CONTINUE HANDLER
FOR NOT FOUND
SET Done=TRUE;

OPEN cur;

read_loop: LOOP

FETCH cur INTO LID;

IF Done THEN
LEAVE read_loop;
END IF;

UPDATE Loans

SET InterestRate=InterestRate+0.5

WHERE LoanID=LID;

END LOOP;

CLOSE cur;

SELECT 'Loan Interest Rates Updated'
AS Message;

END$$

DELIMITER ;
```

Execution Query

```
CALL UpdateLoanInterestRates();
```

Verification Query

SELECT *
FROM Loans;

Output

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Message

▶ Loan Interest Rates Updated

Result 1 x

Result Grid

Filter Rows:

Edits:

Export/Import:

Wrap Cell Content:

Loans 1 x

Output

Action Output

Exercise 7: Packages (MySQL Equivalent)

Note: MySQL does not support Oracle Packages. The following stored procedures and functions are used as the MySQL equivalent.

Scenario 1: CustomerManagement

Requirement:

Add a customer, update customer details, and retrieve customer balance.

Code

```
-- Add Customer
CALL AddCustomer(5,'Kiran','1997-04-10',15000);

-- Update Customer
CALL UpdateCustomer(5,20000);

-- Get Customer Balance
SELECT GetCustomerBalance(5);
```

Execution Query

```
CALL AddCustomer(5,'Kiran','1997-04-10',15000);

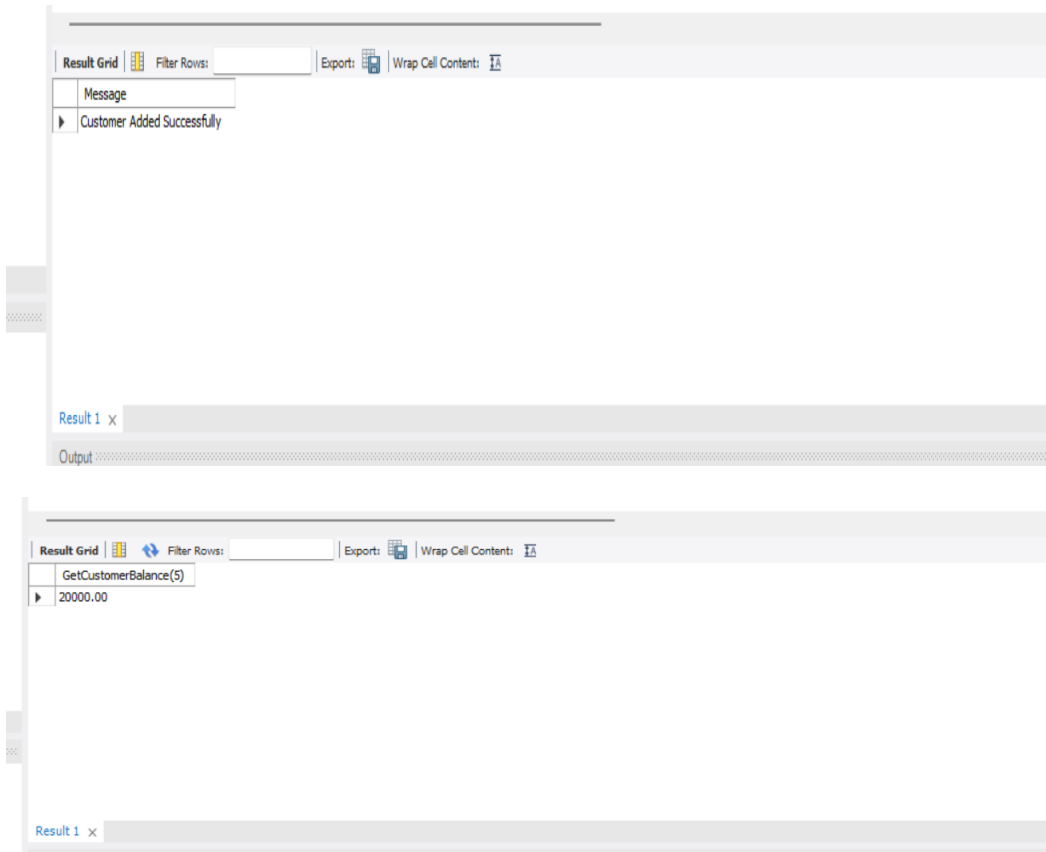
CALL UpdateCustomer(5,20000);

SELECT GetCustomerBalance(5);
```

Verification Query

```
SELECT *
FROM Customers;
```

Output



Scenario 2: EmployeeManagement

Requirement:

Hire an employee, update employee salary, and calculate annual salary.

Code

CALL HireEmployee

```
(  
3,  
'David',  
'Tester',  
50000,  
'QA',  
'2024-01-15'  
);
```

```
CALL UpdateEmployee  
(  
3,  
55000  
);
```

```
SELECT  
EmployeeID,  
Name,  
AnnualSalary(Salary)  
AS AnnualSalary  
FROM Employees;
```

Execution Query

```
CALL HireEmployee(3,'David','Tester',50000,'QA','2024-01-15');
```

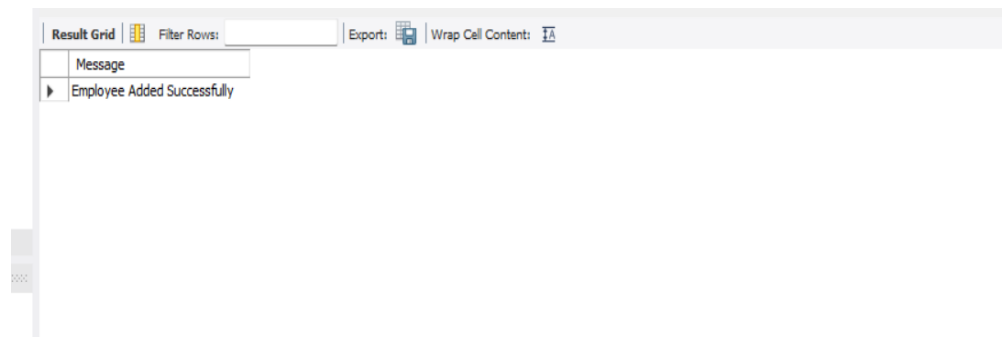
```
CALL UpdateEmployee(3,55000);
```

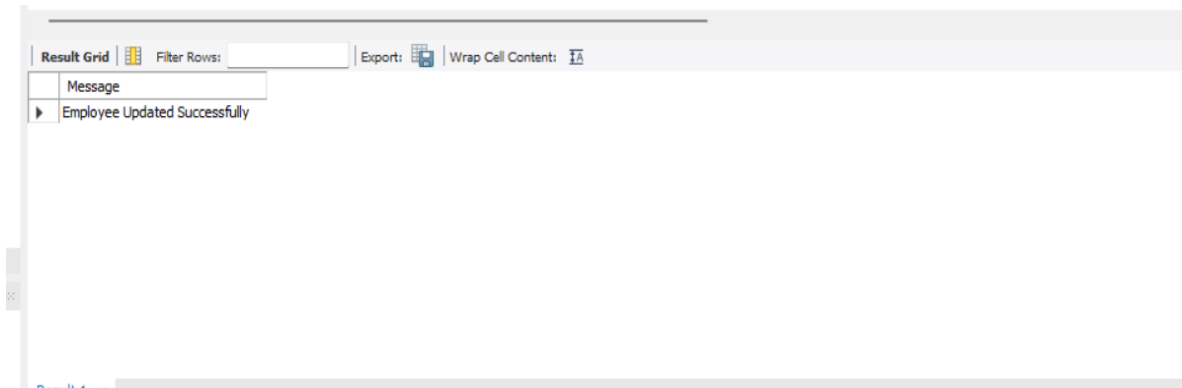
```
SELECT EmployeeID,Name,AnnualSalary(Salary)  
FROM Employees;
```

Verification Query

```
SELECT *  
FROM Employees;
```

Output





Scenario 3: AccountOperations

Requirement:

Open an account, close an account, and calculate the customer's total balance.

Code

CALL OpenAccount

```
(  
  104,  
  5,  
  'Savings',  
  10000  
);
```

CALL CloseAccount

```
(  
  104  
);
```

```
SELECT  
TotalCustomerBalance(1)  
AS TotalBalance;
```

Execution Query

```
CALL OpenAccount(104,5,'Savings',10000);
```

```
CALL CloseAccount(104);
```

```
SELECT TotalCustomerBalance(1);
```

Verification Query

```
SELECT *  
FROM Accounts;
```

Output

The screenshot shows a database application interface with two result grids. The top grid has a header bar with 'Result Grid', a filter row input, and buttons for 'Export' and 'Wrap Cell Content'. Below the header, there is a 'Message' section containing the text 'Account Opened Successfully'. The bottom grid also has a similar header bar. Below its header, there is a single row with the column 'TotalBalance' and the value '10630.00'.

Message
Account Opened Successfully

TotalBalance
10630.00

Verification Queries

```
SELECT * FROM Customers;
```

```
SELECT * FROM Accounts;
```

```
SELECT * FROM Loans;
```

```
SELECT * FROM Employees;
```

```
SELECT * FROM Transactions;
```

```
SELECT * FROM AuditLog;
```

Result Grid  Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 

	LogID	TransactionID	AccountID	Amount	TransactionType	LogDate
▶	1	3	101	1500.00	Deposit	2026-07-02 18:56:00
	2	6	102	500.00	Withdrawal	2026-07-02 18:57:42
✱	NULL	NULL	NULL	NULL	NULL	NULL